

Data Science Tools And Software

Dr. Laila M. Fatehy

Department Of Mathematics And Computer Science
Alexandria University

Reading and Writing Data

Data Formats

1. CSV (Comma Separated Values)

CSV files are plain text files that use specific structuring to arrange tabular data.

Loading a CSV

```
import pandas as pd

# Load CSV into DataFrame
df = pd.read_csv('path_to_your_file.csv')
df
```

Saving to CSV

```
# Save DataFrame to CSV
df.to_csv('output.csv', index=False)
```

2. JSON (JavaScript Object Notation)

- JSON (short for JavaScript Object Notation) has become one of the standard formats for sending data by HTTP request between web browsers and other applications.
- It is a much more flexible data format than a tabular text form like CSV.
- JSON is a lightweight data interchange format that is easy for humans to read and write, and easy for machines to parse and generate.

Loading a JSON file

```
# Load JSON into DataFrame
df = pd.read_json('path_to_your_file.json')
df
```

Note that the nature of the JSON format is hierarchical, so the structure of the JSON file can vary. When loading JSON data, it's important to ensure that the structure matches the expected format.

2. JSON (JavaScript Object Notation)

Saving to JSON

```
# Save DataFrame to JSON
df.to_json('output.json', orient='records')
```

- The `orient` parameter specifies the format of the output:
 - `'split'` : dict like `{index -> [idx], columns -> [columns], data -> [[values]]}`
 - `'records'` : list like `[{column -> value}, ... , {column -> value}]`
 - `'index'` : dict like `{index -> {column -> value}}`
 - `'columns'` : dict like `{column -> {index -> value}}`
 - `'values'` : just the values array

2. JSON (JavaScript Object Notation)

Example

- JSON basic types are objects (dicts), arrays (lists), strings, numbers, booleans, and nulls.
- All of the keys in an object must be strings.
- There are several Python libraries for reading and writing JSON data.
we will use `json` here as it is built into the Python standard library.
- To convert a JSON string to Python form, use `json.loads`

```
# JSON data
obj = """
{"name": "Wes",
"places_lived": ["United States", "Spain", "Germany"],
"pet": null,
"siblings": [{"name": "Scott", "age": 25, "pet": "Zuko"},
{"name": "Katie", "age": 33, "pet": "Cisco"}]
}
"""
```

```
import json
result = json.loads(obj)
result
```

- To show the result as a frame:

```
df1=pd.DataFrame(result['siblings'], columns=['name', 'age'])
df1
```

```
df2=pd.DataFrame(result['places_lived'])
df2
```

- On the other hand `json.dumps` converts a Python object back to JSON

```
asjson = json.dumps(result)
asjson
```

3. Excel

Excel files are commonly used for storing and manipulating data, especially in corporate settings.

Loading an Excel file

```
# Load Excel into DataFrame
df = pd.read_excel('path_to_your_file.xlsx', sheet_name='Sheet1')
df
```

- The `sheet_name` parameter can be a string to specify the name of the sheet or an integer for the position (0-indexed).

Saving to Excel

```
# Save DataFrame to Excel
df.to_excel('output.xlsx', index=False, sheet_name='Sheet1')
```

3. Excel

The `openpyxl` library is required by pandas to handle Excel files.

To resolve this issue, you need to install the `openpyxl` package. You can do this using conda:

```
conda install openpyxl
```

Once `openpyxl` is installed, you should be able to export your DataFrame to an Excel file without any issues.

4. Parquet

Parquet is a columnar storage file format optimized for use with big data processing frameworks like Apache Spark and Apache Hive.

Loading a Parquet file

```
# Load Parquet into DataFrame
df = pd.read_parquet('path_to_your_file.parquet')

print(df)
```

- `pandas` has built-in support for reading and writing parquet files, but you may need to install the `pyarrow` or `fastparquet` library if it's not already installed.

```
conda install pyarrow
```

4. Parquet

Saving to Parquet

```
# Save DataFrame to Parquet  
df.to_parquet('output.parquet', compression='snappy')
```

- The `compression` parameter specifies the type of compression used for the file. Common options include `'uncompressed'`, `'snappy'`, and `'gzip'`.
- There are other variations of the `to_parquet` method that allow you to specify additional options, such as the file format version and the encoding of the data.

5. SQL (Structured Query Language)

SQL is a domain-specific language used in programming and managing data held in a relational database management system (RDBMS). `conda install sqlalchemy`

Connecting to a SQL Database

```
import pandas as pd
from sqlalchemy import create_engine

# Create a connection to the database
engine = create_engine('sqlite:///path_to_your_database.db')

# Load data from SQL into DataFrame
df = pd.read_sql('SELECT * FROM your_table', engine)
```

- `create_engine` creates a connection to the database. The connection string `'sqlite:///path_to_your_database.db'` specifies the type of database (SQLite in this case) and the path to the database file.
- `pd.read_sql` reads the SQL query result into a DataFrame.

Evaluation Metrics

Evaluation Metrics Review

- In predictive modeling, it's crucial to evaluate the performance of your models accurately.
- Different metrics are used depending on the type of problem (classification vs regression).

Recall:

True Positive (TP): Correctly predicted positive instances.

True Negative (TN): Correctly predicted negative instances.

False Positive (FP): Incorrectly predicted positive instances.

False Negative (FN): Incorrectly predicted negative instances.

1. **Confusion Matrix:** A table used to describe the performance of a classification model.

- It shows the number of correct and incorrect predictions.
- It is a useful tool for evaluating the performance
- When there are two classes, which we will call positive and negative (or simply + and –), the confusion matrix consists of four cells, which can be labelled TP, FP, FN and TN as in Figure

		Predicted class		Total instances
		+	–	
Actual class	+	TP	FN	P
	–	FP	TN	N

Classification Metrics

2. **Accuracy:** The proportion of correctly classified instances.

$$\text{Accuracy} = \frac{\text{True Positives} + \text{True Negatives}}{\text{Total Instances}}$$

3. **Precision:** The proportion of true positive predictions among all positive predictions.

$$\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}}$$

4. **Recall (Sensitivity):** The proportion of true positive predictions among all actual positives.

$$\text{Recall} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}}$$

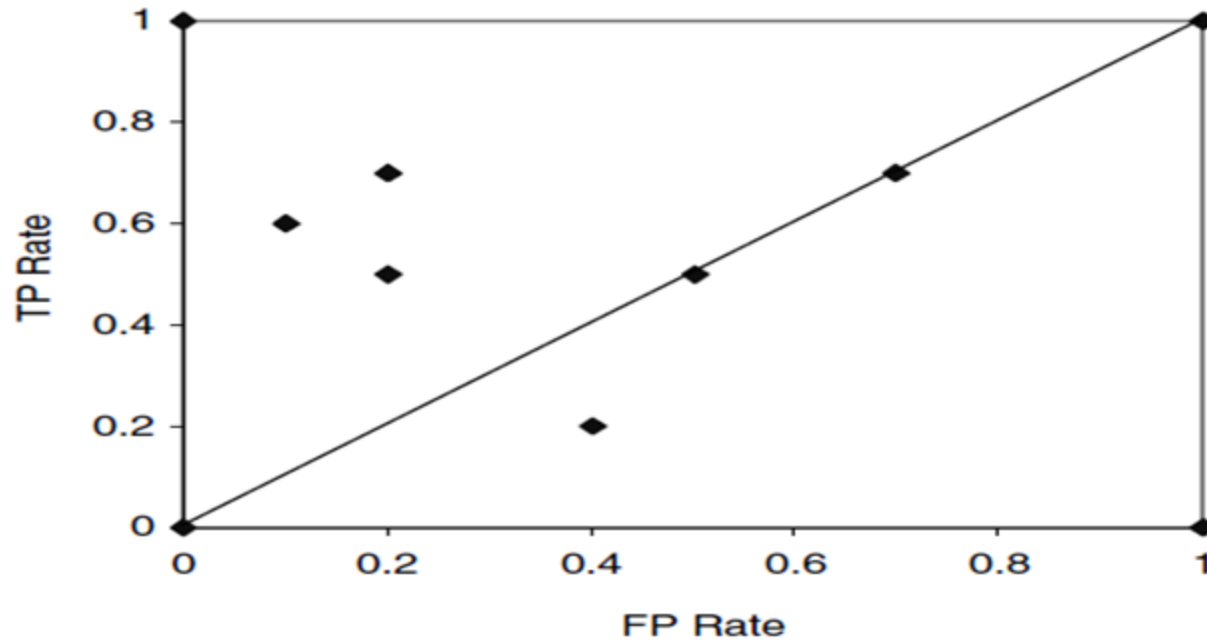
5. **F1-Score:** The harmonic mean of precision and recall.

$$\text{F1-Score} = 2 \times \left(\frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \right)$$

Classification Metrics

6. ROC Curve : Receiver Operating Characteristic curve.

- ROC curve plots True Positive Rate (TPR) against False Positive Rate (FPR).



- EUC provides a single measure of model performance, the smaller the value of Euc the better the classifier

$$EUC = \sqrt{fprate^2 + (1 - tprate)^2}$$

Regression Metrics

1. **Mean Absolute Error (MAE):** Average of absolute differences between predicted and actual values.

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

2. **Mean Squared Error (MSE):** Average of squared differences between predicted and actual values.

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

3. **Root Mean Squared Error (RMSE):** Square root of MSE.

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

Regression Metrics

4. **R-Squared:** Proportion of variance in the dependent variable that is predictable from the independent variables.

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

where:

- y_i are the actual values,
- $\hat{y}_i$ are the predicted values,
- $\bar{y}$ is the mean of the actual values.

Interpretation of R-Squared Metric

- **R-Squared value of 0:** Indicates that the model explains none of the variability of the response data around its mean.
- **R-Squared value of 1:** Indicates that the model explains all the variability of the response data around its mean.
- **R-Squared value between 0 and 1:** Indicates the proportion of the variance in the dependent variable that is predictable from the independent variable(s).